

Aula 6 (Módulo 01) — Rede no Linux: sockets, conexões e firewall

Esta aula dá uma primeira visão prática de rede no Linux. A teoria completa de redes (IP, portas, TCP/UDP, DNS etc.) é o **Módulo 03**, que vem logo em seguida. Aqui o objetivo é já saber rodar os comandos básicos de investigação de rede em um host Linux.

1. Socket e conexão (visão simples)

Um **socket** é o "ponto de encontro" que um programa usa para enviar ou receber dados pela rede — pense nele como uma tomada elétrica: o programa "pluga" nela para se comunicar. Cada socket tem um **endereço IP** (qual máquina) e uma **porta** (qual "canal" específico naquela máquina) associados.

Uma **conexão** existe quando dois sockets — um do seu host, outro do host remoto — estão "falando" um com o outro.

(Vamos aprofundar IP, porta, TCP/UDP com muito mais detalhe no Módulo 03 — por ora, o suficiente é: **IP identifica a máquina, porta identifica o serviço/canal dentro dela.**)

2. `ss` — vendo conexões e portas abertas

`ss` (*socket statistics*) é o comando moderno para listar sockets/ conexões no Linux (substituiu o antigo `netstat`, que você ainda pode ver mencionado em materiais mais antigos).

```
ss -tulpn
```

Decompondo as flags: - `-t` → TCP - `-u` → UDP - `-l` → apenas conexões **listening** (esperando conexão, ou seja, atuando como servidor — lembra da Aula 3 do Módulo 00?) - `-p` → mostra o **processo** dono daquele socket (requer `sudo` para ver processos de outros usuários) - `-n` → mostra números em vez de tentar resolver nomes (mais rápido, e evita fazer consultas DNS desnecessárias)

Saída típica (simplificada):

State	Local Address:Port	Peer Address:Port	Process
LISTEN	0.0.0.0:22	0.0.0.0:*	users:(("sshd",pid=812,fd=3))
LISTEN	127.0.0.1:3306	0.0.0.0:*	users:(("mysqld",pid=1044,fd=21))

Leitura: a porta `22` (SSH) está escutando em **todos** os endereços (`0.0.0.0` = qualquer interface de rede, acessível de fora), enquanto a porta `3306` (MySQL) escuta só em `127.0.0.1` (localhost — só acessível de dentro da própria máquina, não de fora).

Por que importa para segurança: essa diferença (`0.0.0.0` vs. `127.0.0.1`) é crítica. Um banco de dados escutando em `0.0.0.0` em vez de `127.0.0.1` significa que ele está **exposto à rede/internet**, quando o esperado seria só o próprio servidor conseguir acessá-lo — uma das causas mais comuns de vazamento de dados reais é exatamente esse tipo de serviço mal configurado, exposto sem necessidade.

Para ver conexões **ativas** (não só o que está escutando), remova o `-l`:

```
ss -tpn
```

Isso mostra pares `Local Address:Port` ↔ `Peer Address:Port` — ou seja, quem seu host está conversando **agora**, e qual processo é o dono daquela conversa.

3. `ip` — endereço e interfaces de rede

```
ip addr show      # (ou "ip a") mostra as interfaces de rede e seus IPs
ip route show     # (ou "ip r") mostra as rotas — para onde o tráfego é enviado
```

`ip addr` responde "qual é o meu próprio IP, em quais interfaces?" — uma pergunta básica antes de qualquer investigação de rede: você precisa saber qual é o "normal" da própria máquina antes de procurar o que é anômalo.

4. Firewall — primeira visão

O **firewall** decide quais conexões são permitidas ou bloqueadas, com base em regras (ex.: "permitir porta 22 só do IP tal", "bloquear tudo por padrão"). No Linux, a tecnologia por trás disso hoje é o **nftables** (sucessor do antigo `iptables`), que ainda é muito usado e encontrado em ambientes existentes). Ferramentas mais simples de gerenciar, como `ufw` (Ubuntu) ou `firewalld` (RHEL/Fedora), existem por cima dessas tecnologias para facilitar o dia a dia.

```
sudo ufw status verbose    # Ubuntu/Debian
sudo firewall-cmd --list-all  # RHEL/Fedora
```

Vamos aprofundar firewall de verdade (incluindo regras, IDS/IPS) no **Módulo 04 — Network Security**. Por agora, o importante é saber que existe essa camada de decisão **antes** mesmo de o tráfego chegar a um socket/processo.

5. Conectando tudo

```
ip addr      → qual é o meu próprio IP (linha de base)
ss -tulpn    → o que está ESCUTANDO nesta máquina, e qual processo é o dono
ss -tpn      → quem está CONECTADO agora, com quem
firewall     → decide o que é permitido chegar até esses sockets
```

Essas quatro camadas de pergunta são o roteiro básico de qualquer "triagem de rede" em um host Linux.

6. Exercício prático

```
ip addr show
ss -tulpn | head -15
```

Tente identificar: quantas portas estão escutando? Alguma delas escuta em `0.0.0.0` (acessível de fora) quando talvez devesse escutar só em `127.0.0.1`? Não se preocupe em ter certeza absoluta agora — o objetivo é só treinar o olhar. Vamos formalizar "o que é normal vs. suspeito" em rede com muito mais profundidade no Módulo 03 e no Módulo 04.

7. Recapitulando

- Socket = ponto de comunicação de rede (IP + porta); conexão = dois sockets "conversando".
- `ss -tulpn` mostra o que está escutando (servidor) e qual processo é o dono — `0.0.0.0` (exposto) vs. `127.0.0.1` (só local) é uma diferença crítica de segurança.
- `ss -tpn` (sem `-l`) mostra conexões ativas — quem está conectado a quem, agora.
- `ip addr` / `ip route` mostram os próprios endereços e rotas do host.
- Firewall (nftables/iptables, `ufw` / `firewalld`) decide o que é permitido chegar antes mesmo de alcançar um socket.

Próxima aula (fecha o Módulo 01)

Consolidação: os comandos de investigação essenciais reunidos — `grep`, `awk`, `sed`, `find`, `lsof` — as ferramentas de "filtrar, transformar e localizar" que você vai usar em praticamente toda investigação daqui para frente.

Fontes recomendadas

- **Linux man-pages** — `ss(8)`, `ip(8)`: referência oficial de cada comando. <https://man7.org/linux/man-pages/man8/ss.8.html>
- **nftables — documentação oficial (Netfilter Project)**: referência do firewall padrão em distribuições Linux modernas. https://wiki.nftables.org/wiki-nftables/index.php/Main_Page